Multimodal Deep Learning

Solutions to Exercise Sheet 7

Date: 1st of July, 2025

Exercise 1: Selective Parameter Fine-Tuning

(a) **Fishmask** A layer contains 4.0×10^6 weights. A one-shot Fisher score approximation assigns each weight a scalar importance I_i. You may update at most $\mathbf{0.8}\%$ of the weights.

(i) Write the formula that maps the vector of scores $\mathbf{I}$ to a binary mask $\mathbf{m} \in \{0, 1\}^n$ respecting the budget constraint.

(ii) To update the selected weights, the Adam optimizer requires memory for more than just the weights themselves. For each trainable parameter, memory must be allocated for its **gradient**, its **1st moment**, and its **2nd moment**. Assuming all values are stored in 32-bit float precision (4 bytes per number), what is the total memory overhead for the optimizer?

(b) **DiffPruning**

You fine-tune a 350 M-parameter model, targeting a sparsity of $\mathbf{99.5}\%$.

(i) Compute the GPU memory overhead (in MB) of storing the *dense* differentiable mask during training.

(ii) If you could store the final mask as a packed 1-bit tensor for inference, what would its size be (in kB)?

(c) **BitFit**

A GPT-style model has 40 transformer blocks with the following per-block sizes:

- Query bias: $d_{\text{model}} = 768$
- Middle-MLP bias: $d_{\text{ff}} = 3072$

(i) Calculate the *percentage* of the model's 250 M parameters that BitFit updates.

(ii) State one reason why BitFit's advantage fades on very large models or on architectures like LLaMA.

Exercise 2: Adapters

(a) You are adding adapter modules to a transformer with a hidden dimension of $d_{\text{model}} = 768$. Each adapter is a bottleneck feed-forward network with a reduction factor, such that the inner dimension is $r = 16$. The adapter structure is:

$$\text{Linear}(d_{\text{model}} \to r) \to \text{GELU} \to \text{Linear}(r \to d_{\text{model}})$$

(i) Calculate the number of trainable parameters in a **single** adapter module. Remember to include bias terms for both linear layers.

(ii) Explain why this method, despite its parameter efficiency, significantly increases inference latency compared to a method like $(\mathrm{IA})^3$.

(b) $\mathbf{(IA)^3}$

You are fine-tuning the same 40-block, 250 M-parameter model from the BitFit exercise using $(\mathrm{IA})^3$. This method introduces three learnable scaling vectors per transformer block:

- $\mathbf{l_v}$ and $\mathbf{l_k}$ to scale the value and key activations, each a learnable vector of length $d_{\text{model}} = 768$.
- $\mathbf{l_{ffn}}$ to scale the output of the first MLP layer, a learnable vector of length $d_{\text{ff}} = 3072$.

(i) Calculate the total number of trainable parameters introduced by $(\mathrm{IA})^3$ for the entire 40-block model.

(ii) Based on its mechanism, why is $(\mathrm{IA})^3$ considered to have more **limited expressibility** than a standard adapter module?

Exercise 3: Multi-Modal Fusion Strategies

(a) **Representation Fusion** You are predicting house prices using two modalities: an image (A) and a text description (B). After passing through individual encoders, you have the feature vectors:

$$\mathbf{x}_A = \begin{bmatrix} 2 \\ 8 \\ 1 \\ 3 \end{bmatrix} \in \mathbb{R}^4 \quad \text{and} \quad \mathbf{x}_B = \begin{bmatrix} 4 \\ 1 \\ 5 \end{bmatrix} \in \mathbb{R}^3$$

1. **Concatenation Fusion:** Compute the joint representation $\mathbf{z}_{\text{concat}}$ and state its dimension.
2. **Additive Fusion:**
   (a) Can you directly apply additive fusion ($\mathbf{z} = \mathbf{x}_A + \mathbf{x}_B$)? Why or why not?
   (b) To enable additive fusion, you must project both vectors into a common dimension $d_z = 2$ using matrices $\mathbf{W}_A \in \mathbb{R}^{2\times 4}$ and $\mathbf{W}_B \in \mathbb{R}^{2\times 3}$. Write the complete formula for the fused representation $\mathbf{z}_{\text{add}}$.
3. **Bilinear Fusion:**
   (a) Compute the joint representation matrix $\mathbf{Z}_{\text{bilinear}}$ using the outer product ($\mathbf{Z} = \mathbf{x}_A\mathbf{x}_B^T$).
   (b) What is the shape of $\mathbf{Z}_{\text{bilinear}}$? How do its rows and columns relate to the input modalities?
4. Why is Bilinear Fusion not directly applicable for combining three or more modalities simultaneously? What more general fusion method overcomes this?

(b) **Prediction Fusion** A content moderation system uses three separate models (Text, Image, Audio) to detect hate speech. For one post, they output the following probabilities of hate speech:

$$p_{\text{text}} = 0.90 \quad p_{\text{image}} = 0.60 \quad p_{\text{audio}} = 0.15$$

1. *Simple rules.*
   (a) Using plain averaging and threshold 0.5, is the post flagged?

   (b) Apply majority *voting* with a 0.7 modality-specific threshold (1 = hate, 0 = non-hate). Give the final binary decision.

2. *Weighted averaging.* Suppose validation AUROC scores are 0.93 (text), 0.78 (image) and 0.70 (audio).

   (a) Propose weights proportional to AUROC, normalised to sum 1.
   (b) Compute the weighted probability and state the binary outcome at threshold 0.5.
   (c) Discuss one advantage of weighting over unweighted averaging.

3. *Probabilistic fusion.* Under an "AND" logic (multiply probabilities) calculate the joint probability and comment on when such a rule may be preferable despite lower recall.
4. *Model-based post-processing.* Outline how you would prompt an LLM to reconcile disagreeing modalities. Specify what information you would feed (e.g. class labels, confidence scores, textual rationale snippets).

Exercise 4: Exercise: Multimodal Fusion for Radiology Reporting

Scenario A system receives three inputs for a single patient:

1. **Chest X-ray**: A single high-resolution 2D radiograph (e.g., 1024×1024 pixels).
2. **Patient meta-data**: Semi-structured information (e.g., `Age: 65; Sex: M;`).
3. **Referrer note**: Unstructured free text from the referring physician.

The goal is to autoregressively generate a structured radiology report, such as:
`Findings: ... Impression: ...`

Your task is to design a complete transformer-based pipeline for this problem. You will define the tokenization, choose an architecture, propose a training objective, and analyze its computational efficiency.

(a) **Input Tokenisation and Representation**

   i. **X-Ray Image Tokens** The raw image must be converted into a sequence of feature vectors. Outline one *soft token* strategy and one *hard token* strategy.
   ii. **Pros and Cons** For this specific radiology task, summarize two key pros and cons of your proposed soft vs. hard tokenization strategies. (Hint: Think about reconstruction fidelity, semantic abstraction, and the granularity of visual information).
   iii. **Text and Metadata Tokens** Propose a unified tokenization strategy for both the patient meta-data and the referrer note. How would you ensure the model can distinguish between these two types of textual information in the joint input sequence?
   iv. **Input Sequence Sketch** Sketch the final concatenated input sequence for your model. Clearly mark the boundaries between different modalities and indicate the embedding process for each.

(b) **Multimodal Architecture Design**

After tokenizing all inputs into a single sequence, you must choose an architecture to process it and generate the report. Pick **one** of the two architectural paradigms below and justify your selection based on its expected performance, training efficiency, and potential trade-offs.

- **(A) Standard Encoder-Decoder Transformer:**
  - A deep **Encoder** processes the complete multimodal input sequence via self-attention. Every token attends to every other token.
  - A **Decoder** then autoregressively generates the report tokens, using cross-attention to access the encoder's full output at every step.
- **(B) Latent-Query Transformer (Perceiver-style):**
  - An initial **Cross-Attention** layer compresses the long multimodal input sequence into a short, fixed-size array of N learned latent vectors (queries).
  - A deep **Latent Transformer** (processing tower) refines these N vectors using self-attention, decoupled from the input length.
  - A **Decoder** autoregressively generates the report tokens, using cross-attention to access the final N latent vectors.

Explain how the standard training objective for generation: next-token prediction (causal language modeling loss), would be applied to your chosen architecture.

(c) **Training Strategy**

Training your chosen architecture from scratch on a limited number of full radiology reports is unlikely to succeed. A crucial step is large-scale pre-training on more general, unlabeled data to learn fundamental multimodal relationships.

Propose **one** self-supervised pre-training objective to teach your model how to connect vision and text. Describe the objective, the type of data it would require, and why it would be beneficial for the final radiology reporting task. (Hint: Consider objectives discussed in last week's lecture).

(d) **Computational Analysis** Analyze the computational cost of the most expensive component in each architecture during a single forward pass. Assume:

- The input image produces $M_{img} = 4096$ tokens (from 16×16 patches). Text and metadata add $M_{text} = 256$ tokens. Total input tokens $M = 4352$.
- The hidden dimension is $d = 768$.
- For architecture (B), the latent array has a length of $N = 256$, and the processing tower has $L = 12$ layers.
- The approximate FLOPs for an attention layer are:
  - Self-Attention on a sequence of length S: $FLOPs \approx 2 \cdot S^2 \cdot d$
  - Cross-Attention with query length N and key/value length M: $FLOPs \approx 2 \cdot N \cdot M \cdot d$

i. **Cost of Architecture A** The bottleneck in the Standard Encoder-Decoder is the encoder's self-attention. Calculate the FLOPs for **one** self-attention layer in the encoder, which processes the full sequence of M tokens.

ii. **Cost of Architecture B** The Latent-Query model has two main computational steps: (i) the initial cross-attention layer to compress the M input tokens into N latents, and (ii) the L self-attention layers on the N latents. Calculate the total FLOPs for these components.

iii. **Comparison** Which design is computationally cheaper, and by what approximate factor?

iv. **Break-Even Point** Derive the equation for the input length M_{eq} at which the computational cost of one self-attention layer in Architecture A's encoder is equal to the cost of the initial cross-attention layer in Architecture B. Solve for M_{eq} using the given parameters. What does this value tell you?